

Extra Content: Combining CTD and Discrete Datasets

Background

For your field course projects, you will likely need to combine CTD data with the discrete data. Here, we will step through this process, using the `DaRTS_CTD_data.csv` and `DaRTS_discrete_data.csv` datasets as a starting point.

How to approach merging the datasets

There are multiple ways to merge the datasets depending on the exact combination of variables you want e.g. do you want all the stations and/or all the depths and/or only one station and/or multiple years, etc. Here, we want to merge all our data.

To figure out the way to do the merge, we need to break down the process into the most basic/small steps. This is good practice for any kind of code development: breaking down your task into discrete, bitesize bundles. Think about building a robot to make a sandwich. You couldn't just code "make a sandwich", you'd need to break it down into steps, like a recipe e.g. go to the cupboard, take out the bread, put it on the table, etc. Some of those steps may seem obvious to you, but it wouldn't be to a robot.

1. *What* general data formatting steps do you need to take?

We need to match the discrete and CTD data based on key identifying information. In this case, that information is:

1. The cruise date (for the DaRTS data, the different cruises are identified by the date),
2. The station number,
3. Depth (depending on your exact end goal / question, you'll need to think a bit carefully about how we want to match by depth).

Therefore, we're going to need to make sure those data are all represented the same way in each of our datasets, so we can match data between the two datasets.

2. Make a step-by-step plan

With the above information, we can make a step-by-step plan for the data *wrangling* (or formatting) that we need to do. Sometimes one of these steps ends up getting broken down into even more steps at a later point when you realise something may not be as straightforward as you first thought.

Step-by-step plan:

1. Import CTD data from csv
2. Import discrete data from csv
3. Check and reformat dates in each dataset if necessary

4. Check and reformat depths in each dataset if necessary
5. Combine (merge) the discrete and CTD data

Now we have our plan, we can start doing it!

Importing the data sets

Let's load each of our datasets into R. But first... we need to start our R session by

1. Setting our working directory
2. Importing the packages we need for today

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
library(ggplot2)
```

```
## Warning: package 'kableExtra' was built under R version 4.5.3

##
## Attaching package: 'kableExtra'

## The following object is masked from 'package:dplyr':
##
##   group_rows
```

Ok, so now we are ready to import our data sets. We are going to load each one into a separate data frame:

```
ctdData <- read.csv('../data/DaRTS_CTD_data.csv')
discreteData <- read.csv('../data/DaRTS_discrete_data.csv')
```

Combining data sets

In our plan, we identified the important columns to merge on (cruise, date and depth), and we decided we needed to check the format of the date and depth data in each of our datasets.

Dates

We know one of our key identifying pieces of information is date. So let's take a closer look at the dates. First, let's pull out just the dates column:

```
# one way
ctdDates <- ctdData$date

# another way:
ctdDates <- ctdData[['date']]

# and the discrete dates
discreteDates <- discreteData$date
```

In the above two ways of pulling out the dates column we ended up with a vector object. You can also pull out the dates column and have it as a 1 column data frame:

```
ctdDates_df <- ctdData['date']
```

What format are these dates in? You can check by manually opening the two data frames (`ctdData` and `discreteData`) and looking at the Dates column (i.e. double click on the data frames in the Environment Panel). Or we can check using R:

```
ctdDates[1]
```

```
## [1] 20120911
```

What type of data is this? A number?

```
class(ctdDates[1])
```

```
## [1] "integer"
```

Numbers that do not contain decimal values have a data type as an integer.

What about the discrete data dates?

```
discreteDates[1]
```

```
## [1] "9/11/2012"
```

```
class(discreteDates[1])
```

```
## [1] "character"
```

A character can be a letter/number or a combination of letters and/or numbers enclosed by quotes.

Note that for the discrete data in 2025 the dates are in a different format (maybe you noticed this in the spreadsheet):

```
discreteDates[1051]
```

```
## [1] "20251104"
```

```
class(discreteDates[1051])
```

```
## [1] "character"
```

While the format is different, the date is still saved as a character. We'll need to keep this in mind for later.

So in each dataset, the dates are different formats and different data types. Let's make them the same format and data type - this will make things much easier for matching the discrete and CTD data.

Should we have our dates represented by numeric values? Or characters?

If we use characters, we need to be careful about knowing which part of the date is the month, day and year: in the US it's normal for dates to be month/day/year, but in almost every other country it's day/month/year. This can lead to a lot of confusion when working collaboratively with international participants, or even within a research group if someone from Scotland joins a team of Americans! In addition to the confusion about the day-month-year order, when we use characters, it's really difficult to do any date math e.g. counting how many days between two dates.

This may make it seem like numbers are a better option - but this can still lead to difficulties when wanting to do date math, e.g. if you want to know how many days there are between dates that span multiple months, you need to know how many days are in each of those months, and if one of those months is February, are you in a leap year?

As you can see, we need to approach dates (& times) carefully. Most programming languages have their own class of data for dates & times (i.e. not numeric or characters). So a lot of those difficulties I mentioned are dealt with behind the scenes by the different date math functions that exist.

R deals with dates and times in a couple of different ways. We're going to use the `lubridate` package which is part of the `tidyverse`. There's this great cheat sheet which gives an overview of a lot of the functionality of the `lubridate` package.

Date reformatting

Let's first load the `lubridate` package

```
library("lubridate")
```

```
##
```

```
## Attaching package: 'lubridate'
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
##     date, intersect, setdiff, union
```

Now, let's deal with the CTD dates. We can use the `ymd` function to tell `lubridate` that our date is currently a number stored in the format `yyyymmdd`.

```
ctdDates <- lubridate::ymd(ctdData$date)
```

What do these look like?

```
ctdDates[1]
```

```
## [1] "2012-09-11"
```

And what data type are they?

```
class(ctdDates)
```

```
## [1] "Date"
```

OK great - now what about the discrete dates? We can't use the same approach as above because our discrete dates are characters of the form mm/dd/yyyy or yyyyymmdd. We can use something similar:

```
discreteDates <- lubridate::as_date(lubridate::parse_date_time(discreteData$date, orders = c("mdy", "ymd")))
```

Here we're asking lubridate to do as it did for the CTD data, but first to parse the date and time i.e., check if the date is of the form mm/dd/yyyy, and if not, then check the data is of the form yyyyymmdd, and then convert those to the date class.

Let's just check what the dates that were successfully converted look like and their type:

```
discreteDates[1]
```

```
## [1] "2012-09-11"
```

```
class(discreteDates)
```

```
## [1] "Date"
```

Great - now we have nicely formatted dates that we can use later to match our CTD and discrete data together. To make this task easier, we want to include a new column in each of our data frames that contains these newly formatted dates. To do this, we are going to use **dplyr**'s **mutate** function. We used **filter** from **dplyr** last day. **dplyr** is another part of the **tidyverse** that is for data frame manipulation.

Adding new columns to a data frame with mutate

In the following, we are going to add a new column to our data frames with these newly formatted dates. In each case, the new column will have the name **Date**. Note if a column already exists with the name you are assigning in the mutate function, the original column will be overwritten. So here, we are overwriting the original dates, with the newly formatted dates.

```
ctdData <- mutate(ctdData, date = ymd(ctdData$date))
```

```
discreteData <- mutate(discreteData, date = lubridate::as_date(lubridate::parse_date_time(discreteData$date, orders = c("mdy", "ymd"))))
```

To do any type of data frame manipulations, you can use *pipes*. Pipes are a recent addition to R. They let you take the output of one function and send it directly to the next, which is useful when you need to do many things to the same dataset. Pipe has the symbol `%>%`. We'll make full use of pipes later, but to add the dates column to the `ctdData` data frame using pipes you would do:

```
ctdData <- ctdData %>% mutate(date = ymd(ctdData$date))
```

This is equivalent to `ctdData <- mutate(ctdData, Date = ymd(ctdData$Date))`.

Tip for working with dates I recommend including in your datasets separate columns for the year, month and day, as well as one column for the combined date. This is really useful when you start to do your analysis and you want to look at the data from one year, or one month or even one day! Let's do that now for our data

```
ctdData <- ctdData %>%  
  mutate(year = year(ctdData$date),  
         month = month(ctdData$date),  
         day = day(ctdData$date))  
  
discreteData <- discreteData %>%  
  mutate(year = year(discreteData$date),  
         month = month(discreteData$date),  
         day = day(discreteData$date))
```

The other useful way to represent the date is day of year, it can be particularly useful for certain analyses. Let's also include a day of year column

```
ctdData <- ctdData %>%  
  mutate(doy = yday(ctdData$date))  
  
discreteData <- discreteData %>%  
  mutate(doy = yday(discreteData$date))
```

Depths

The depth is another key piece of identifying information for merging our datasets.

What do the depths look like for each dataset?

Let's print out the first 10 depths of the CTD data:

```
ctdData$depth_m[1:10]
```

```
## [1] 1.363 1.346 1.345 1.354 1.360 1.362 1.355 1.342 1.342 1.356
```

And of the discrete data:

```
discreteData$depth[1:10]
```

```
## [1] 2 10 30 2 10 40 2 10 40 2
```

The discrete measurements are at well defined depth intervals, but the CTD measurements were continuous - so how are we going to match these?

We need to make some assumption - here, we're going to *bin* the CTD data to the nearest meter. That is, we are going to take all the data within a depth interval (also known as a bin), and take the mean value of the data within that bin. Here, our depth interval will be 1 m, i.e. we'll bin our CTD data such that we end up with data at depths of 1, 2, 3, 4, 5, ... meters.

Binning using `group_by` and `summarize`

This process of binning data is an example of the split-apply-combine paradigm: split the data into groups, apply some analysis to each group, and then combine the results. Many data analysis tasks can be approached using this technique.

In the case of binning:

- split the data into groups: we're splitting our data into groups based on the depth.
- apply some analysis to each group: we're taking the mean of the data in each of our groups
- combine the results: we're putting all the mean data back into a new dataframe.

Here, we're going to make use of pipes too.

First, we need to round our depths to the nearest meter, then we can bin them.

```
ctdData <- ctdData %>% mutate(depth_m = round(ctdData$depth_m))
```

Now for the binning.

```
ctdDataBinned <- ctdData %>%  
  group_by(depth_m) %>%  
  summarize_all(mean)
```

```
## Warning: There were 224 warnings in 'summarise()'.  
## The first warning was:  
## i In argument: 'ph = (function (x, ...) ...'.  
## i In group 1: 'depth_m = 0'.  
## Caused by warning in 'mean.default()':  
## ! argument is not numeric or logical: returning NA  
## i Run 'dplyr::last_dplyr_warnings()' to see the 223 remaining warnings.
```

Let's take a look at our final dataframe, we'll open it from the Environment Pane. On first glance, this looks like it's done what we want - we've only got depths of rounded meters. But, if we look at the other columns, things don't look quite right. The stations are no longer sensible station numbers, and there's quite a lot of data missing.

What we've actually done is calculated the mean profile for all the CTD data together, *rather than the mean profile for the CTD data at every station on every cruise*. So we need to split our data using more criteria. Also, when we calculated the mean, our missing data got in the way. We need to tell R to ignore the missing data, and just calculate the mean on the data that are there.

```
ctdDataBinned <- ctdData %>%  
  group_by(date, station, depth_m) %>%  
  summarize_all(mean, na.rm = TRUE)
```

```
## Warning: There were 33602 warnings in 'summarise()'.
## The first warning was:
## i In argument: 'ph = (function (x, ...) ...)'.
## i In group 1: 'date = 2012-09-11', 'station = 1', 'depth_m = 1'.
## Caused by warning in 'mean.default()':
## ! argument is not numeric or logical: returning NA
## i Run 'dplyr::last_dplyr_warnings()' to see the 33601 remaining warnings.
```

Merging the discrete and CTD data

Now we've got dates & depths in the same format in our discrete and CTD datasets, we can merge them together.

Dataframe merging is a common task in many data science applications, and there is some terminology involved to describe the different ways you can merge the dataframes.

Dataframe merging approaches

Mutating joins allow you to combine variables from multiple tables. There are four types of mutating join, which differ in their behaviour when a match is not found. We'll illustrate each with an example. Let's start by making two dataframes:

```
df1 <- data.frame(x = c(1, 2), y = 2:1)
```

x	y
1	2
2	1

```
df2 <- data.frame(x = c(3, 1), a = 10, b = "a")
```

x	a	b
3	10	a
1	10	a

`inner_join(df1, df2)` Only includes observations that match in both df1 and df2.

```
df_innerjoin <- df1 %>% inner_join(df2)
```

```
## Joining with 'by = join_by(x)'
```

x	y	a	b
1	2	10	a

Here, R was clever, and merged on the common column between the two dataframes.

`left_join(df1, df2)` Includes all observations in `df1`, regardless of whether they match or not. This is the most commonly used join because it ensures that you don't lose observations from your primary table.

```
df_leftjoin <- df1 %>% left_join(df2)
```

```
## Joining with 'by = join_by(x)'
```

x	y	a	b
1	2	10	a
2	1	NA	NA

`right_join(df1, df2)` Includes all observations in `df2`. It's equivalent to `left_join(df2, df1)`, but the columns and rows will be ordered differently.

```
df_rightjoin <- df1 %>% right_join(df2)
```

```
## Joining with 'by = join_by(x)'
```

x	y	a	b
1	2	10	a
3	NA	10	a

In this case, we called `df1` first, so it's columns are listed first in the merged dataframe.

```
df2_leftjoin <- df2 %>% left_join(df1)
```

```
## Joining with 'by = join_by(x)'
```

x	a	b	y
3	10	a	NA
1	10	a	2

In this case, `df2` was listed first, so it's columns are first in the merged dataframe.

`full_join()` Includes all observations from `df1` and `df2`.

```
df_fulljoin <- df1 %>% full_join(df2)
```

```
## Joining with 'by = join_by(x)'
```

x	y	a	b
1	2	10	a
2	1	NA	NA
3	NA	10	a

The left, right and full joins are collectively known as **outer joins**. When a row doesn't match in an outer join, the new variables are filled in with missing values.

Back to the discrete and CTD data

What type of join do we want to do here?

We are merging the *whole* of the discrete dataset and the *whole* of the binned CTD dataset. **And** we are merging on date, depth and station. Thus, we want to include *all the observations* from each dataset, then later we can filter out the data we're interested in (e.g., for your field projects). To include all the observations, we need to do a full join:

```
combinedData <- ctdDataBinned %>%
  full_join(discreteData)
```

```
## Joining with 'by = join_by(date, station, depth_m, ph, year, month, day, doy)'
```

Final step is to save our new dataframe as a CSV file.

```
write.csv(combinedData, "../data/DaRTS_combined_data.csv")
```